

Detecting AI-Generated Speech Using Traditional Speech Processing Features

Lara Fuqaha, Taymaa Nasser

Department of Electrical and Computer Engineering, Birzeit University, Palestine

122071@student.birzeit.edu, 1222640@student.birzeit.edu

Abstract

The rapid advancement of text-to-speech and voice-conversion systems has made synthetic speech increasingly indistinguishable from human speech. This project investigates whether traditional speech-processing features can capture acoustic artifacts introduced by such systems, enabling classification of real (bonafide) versus AI-generated (spoof) speech. Using a balanced subset of the ASVspoof 2021 Deepfake evaluation dataset, we extract MFCC combined with spectral features, LFCC, and mel-spectrograms, and train SVM, Random Forest, and CNN classifiers. Our best model, an SVM trained on MFCC and spectral features with proper feature scaling, achieves 92.1 percent test accuracy, outperforming Random Forest, LFCC-based models, and a CNN. A generalization experiment shows accuracy drops from 94.5 to 88.8 percent on unseen spoofing attack types. Results suggest well-engineered traditional features remain highly competitive with deep learning under limited-data conditions.

Index Terms: speech deepfake detection, MFCC, LFCC, spectrogram, SVM, CNN

1. Introduction

The increasing realism of AI-generated speech, driven by advances in text-to-speech (TTS) and voice-conversion technology, poses a growing challenge for voice-based authentication systems, media integrity, and misinformation prevention. While synthetic speech can now closely mimic natural human prosody and timbre, subtle acoustic artifacts often remain in the underlying signal characteristics.

Our objective is to build and evaluate a machine learning pipeline capable of distinguishing bonafide human speech from spoofed speech, using the ASVspoof 2021 DF dataset. We compare multiple feature representations (MFCC, spectral features, LFCC, and mel-spectrograms) and multiple classifiers (SVM, Random Forest, and a CNN), and test how well our best-performing model generalizes to spoofing attack types not seen during training.

1.1. Related work

The ASVspoof challenge series has served as the primary benchmark for spoofing and deepfake detection research since 2015, with the 2021 edition introducing the Deepfake (DF) track targeting speech generated via TTS and voice-conversion systems under varied codec conditions [1]. Traditional countermeasure systems have relied on cepstral features such as MFCC and LFCC combined with classifiers such as GMM or SVM, while recent work has explored end-to-end deep learning approaches operating on raw waveforms or spectrograms [2].

While toolkits such as Praat, Kaldi, and HTK exist for phonetic analysis and speech recognition, this project uses `librosa` [3] for feature extraction due to its lightweight integration with machine learning workflows, and `spafe` [4] for LFCC computation.

2. Methodology

2.1. Dataset

We use the ASVspoof 2021 DF evaluation dataset, which labels each spoofed sample by its specific spoofing algorithm (attack type). The full evaluation partition exhibits severe class imbalance (14,869 bonafide vs. 519,059 spoof samples). To avoid biasing our classifiers, we constructed a balanced subset of 2,000 bonafide and 2,000 spoof samples, stratified by random sampling with a fixed seed for reproducibility.

2.2. Feature extraction

All audio was resampled to 16 kHz and processed using 25 ms frames with a 10 ms hop. Three feature sets were extracted: (1) MFCC + Spectral: 13 MFCC coefficients (mean/std), short-time energy, zero-crossing rate (ZCR), and four spectral features (centroid, bandwidth, rolloff, flux), yielding 38 dimensions; (2) LFCC: 13 linear-frequency cepstral coefficients (mean/std) via `spafe` [4], combined with energy and ZCR, yielding 30 dimensions; (3) Mel-spectrogram: 128-band log-mel spectrograms, padded/truncated to 128 frames, used as 2D CNN input.

2.3. Preprocessing

Feature vectors were normalized using z-score standardization, fit exclusively on training data. This proved critical: an initial unscaled experiment showed spectral features degrading SVM performance (73.5% vs. a 77.4% baseline) due to scale imbalance between small-magnitude MFCC values and large-magnitude spectral features. After normalization, accuracy improved to 92.1%.

2.4. Model training

Data was split 80/20 (train/test, stratified). SVM (RBF kernel) and Random Forest were tuned using 5-fold cross-validated grid search over the training set: SVM's $C \in \{0.1, 1, 10\}$; Random Forest's estimators $\in \{100, 200, 300\}$ and max depth $\in \{\text{None}, 10, 20\}$. The best configuration was refit on the full training set and evaluated once on the held-out test set. All classical models were implemented using `scikit-learn` [5].

The CNN was trained on 128×128 mel-spectrograms using three convolutional blocks (32/64/64 filters with max-pooling),

a dense layer of 64 units with dropout (0.5), and a sigmoid output (858,689 parameters total), trained for 20 epochs with Adam and binary cross-entropy loss on a 60/20/20 split. The CNN was implemented in TensorFlow/Keras [6].

2.5. Generalization test

To assess generalization, two attack types (A16 and A09) were entirely excluded from training. A separate model was trained on the remaining attack types and evaluated both on held-out seen attacks and on the excluded, unseen attack types.

3. Experiments and results

3.1. MFCC + Spectral features

Table 1 and Table 2 report full classification metrics for SVM and Random Forest trained on MFCC+Spectral features.

Table 1: *SVM (MFCC+Spectral) test results*

Class	Precision	Recall	F1
Bonafide	0.91	0.94	0.92
Spoof	0.94	0.90	0.92
Accuracy = 92.1%, AUC = 0.97			

Table 2: *Random Forest (MFCC+Spectral) test results*

Class	Precision	Recall	F1
Bonafide	0.91	0.82	0.86
Spoof	0.83	0.92	0.88
Accuracy = 86.9%, AUC = 0.94			

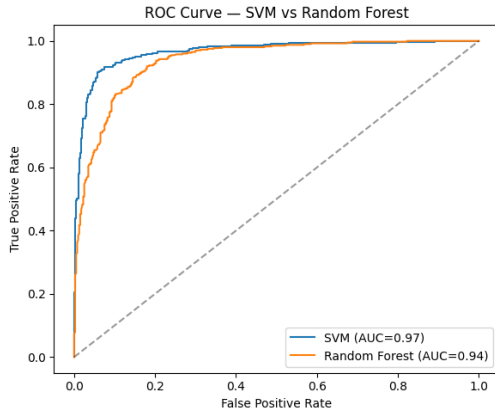


Figure 1: *ROC curves: SVM vs. Random Forest (MFCC+Spectral).*

The SVM achieved the highest accuracy of all models tested, with balanced precision and recall across both classes (Table 1). Random Forest consistently produced more false positives on bonafide speech than SVM (Table 2), and its lower AUC (0.94 vs. 0.97, Fig. 1) confirms weaker overall separability between classes.

3.2. MFCC vs. LFCC

Table 3 and Table 4 report results for the LFCC feature set.

Table 3: *SVM (LFCC) test results*

Class	Precision	Recall	F1
Bonafide	0.91	0.88	0.89
Spoof	0.89	0.91	0.90
Accuracy = 89.6%			

Table 4: *Random Forest (LFCC) test results*

Class	Precision	Recall	F1
Bonafide	0.85	0.82	0.84
Spoof	0.83	0.85	0.84
Accuracy = 83.9%			

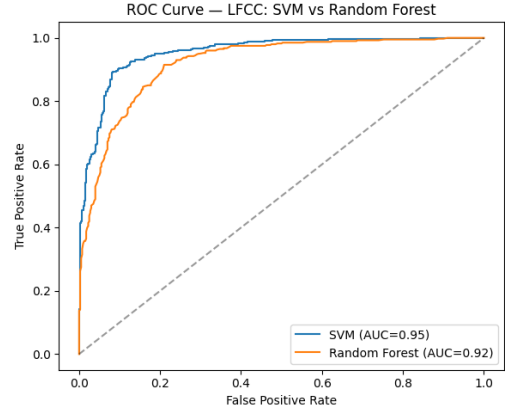


Figure 2: *ROC curves: SVM vs. Random Forest (LFCC).*

MFCC+Spectral outperformed LFCC across both classifiers (SVM: 92.1% vs. 89.6%; RF: 86.9% vs. 83.9%), suggesting the Mel-scale filterbank combined with spectral shape descriptors captures more discriminative synthetic-speech artifacts than the linear-frequency representation alone.

3.3. Classical ML vs. deep learning

Table 5 reports CNN test performance.

Despite the popularity of deep learning for audio classification, the CNN (87.5%) did not outperform the SVM trained on hand-crafted features (92.1%). Fig. 3 shows a widening gap between training accuracy (94.8%) and validation accuracy (88.4%) over epochs, indicating overfitting. With only 2,400 training samples relative to 858,689 parameters, the CNN likely lacked sufficient data to learn generalizable representations directly from raw spectrograms, whereas the SVM benefited from compact, domain-informed feature engineering.

3.4. Generalization to unseen attacks

Table 6 presents the generalization experiment results.

Both models show a performance drop on unseen attack types, confirming that some learned patterns are attack-specific

Table 5: CNN (mel-spectrogram) test results

Class	Precision	Recall	F1
Bonafide	0.91	0.84	0.87
Spoof	0.85	0.91	0.88
Accuracy = 87.5%			

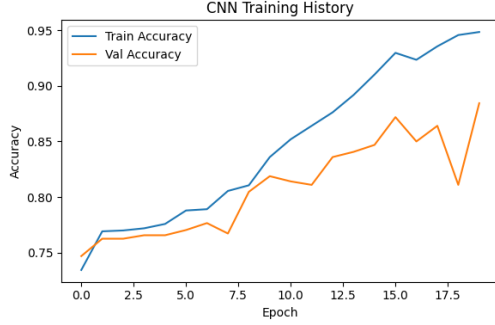


Figure 3: CNN training vs. validation accuracy over 20 epochs.

rather than fully general. SVM shows a larger absolute drop (5.7 points) than Random Forest (2.3 points), despite higher accuracy overall in both conditions, suggesting a trade-off between peak performance and robustness to distribution shift. Examining the unseen-attack confusion matrix for SVM shows the drop is driven primarily by reduced spoof recall (0.95 to 0.81), rather than increased false alarms on bonafide speech (recall improved slightly, 0.94 to 0.96), indicating the model’s weakness lies in detecting unseen synthesis artifacts rather than in over-flagging genuine speech.

4. Team member contributions

Lara Fuqaha: dataset construction, MFCC and spectral feature extraction, SVM and Random Forest training, hyperparameter tuning, and generalization. Taymaa Nasser: Evaluation plots, LFCC feature extraction and library integration, SVM/RF training on LFCC features, CNN architecture design, spectrogram extraction, CNN training and evaluation.

5. Conclusion

This project investigated whether traditional speech-processing features can effectively distinguish bonafide human speech from AI-generated speech. Our best model, an SVM trained on MFCC and spectral features with proper feature normalization, achieved 92.1% test accuracy, outperforming Random Forest, an LFCC-based alternative, and a CNN trained directly on mel-spectrograms. Feature scaling proved to be a critical, non-obvious factor: an initial unscaled configuration performed worse than a simpler baseline, underscoring the importance of careful preprocessing.

Our generalization experiment revealed that model performance degrades moderately when facing previously unseen spoofing attack types, highlighting a genuine limitation and pointing to future work on incorporating a broader diversity of attack types during training. A key limitation of this study is the relatively small subset size (4,000 samples) used due to computational constraints, which may have disadvantaged the CNN

Table 6: Seen vs. unseen attack types (A16, A09)

Model	Seen	Unseen
SVM	94.5%	88.8%
Random Forest	87.2%	84.9%

given its larger parameter count. Future work could explore larger subsets, additional features such as Constant-Q Cepstral Coefficients (CQCC), or data augmentation to improve CNN generalization.

6. Acknowledgments

The authors thank the course instructor for providing the dataset and project guidelines.

7. Generative AI Use Disclosure

Claude (Anthropic) [7] was used to assist with code debugging, pipeline design guidance, and drafting portions of this manuscript. All experimental design decisions and result interpretations were reviewed and validated by the authors.

8. References

- [1] J. Yamagishi, X. Wang, M. Todisco, M. Sahidullah, J. Patino, A. Nautsch, X. Liu, K. A. Lee, T. Kinnunen, N. Evans, and H. Delgado, “ASVspoof 2021: Automatic speaker verification spoofing and countermeasures challenge evaluation plan,” *arXiv preprint arXiv:2109.00535*, 2021.
- [2] J. Frank and L. Schönherr, “WaveFake: A data set to facilitate audio deepfake detection,” *arXiv preprint arXiv:2111.02813*, 2021.
- [3] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in python,” 2015, <https://librosa.org>.
- [4] A. El Amine, “spafe: Simplified python audio features extraction,” 2021, <https://github.com/SuperKogito/spafe>.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015, <https://www.tensorflow.org>.
- [7] Anthropic, “Claude (large language model),” 2025, used for code debugging assistance and manuscript drafting support. <https://www.anthropic.com>.